

Corrine Sadler

D597-Data Management

December 12, 2025

Task 2: Non-Relational Database Design and Implementation

Part 1: Design Document

A. Scenario 2

1. EcoMart is a rising online marketplace focused on providing consumers with access to environmentally conscious and sustainably sourced products across multiple categories, including groceries and personal care items. As EcoMart continues to grow, the business must manage an increasingly diverse and complex set of product data. A traditional relational database is limited in its ability to efficiently support this level of data variability due to its reliance on fixed schemas and predefined table structures. When product attributes change or expand, schema updates are often required in relational systems, increasing maintenance efforts and negatively impacting system efficiency. These limitations restrict EcoMart's ability to scale its platform, maintain optimal performance, and adapt quickly to changing data requirements.
2. A NoSQL database solution is well suited to address EcoMart's business needs because it provides the flexibility and scalability required to manage diverse and evolving product data. NoSQL databases are designed to handle data beyond the constraints of fixed schemas; according to MongoDB, "NoSQL databases are purpose-built for specific data models and have flexible schemas for building modern applications" (MongoDB, 2024). This flexibility allows new product attributes to be added without restructuring existing data, which is particularly important for

EcoMart, where categories such as cosmetics and groceries contain significantly different and independently evolving data fields. In addition, NoSQL databases support horizontal scaling, enabling EcoMart to efficiently distribute data across multiple servers as data volume and user traffic increase. By adopting a NoSQL database solution, EcoMart can improve performance, reduce maintenance complexity, and ensure that the platform remains responsive and adaptable as the marketplace continues to grow.

3. The most appropriate NoSQL database type for EcoMart's marketplace is a document-based database, specifically MongoDB. Document-based databases store data in flexible, JSON-like documents, allowing related attributes to be grouped together within a single record. This structure is particularly well suited for EcoMart because product data varies significantly across categories such as cosmetics and groceries. For example, cosmetic products include detailed ingredient lists, skin-type compatibility indicators, pricing, and product ranks, while grocery data focuses on transactional information such as member identifiers, purchase dates, and item descriptions. MongoDB's document-based design allows each product or transaction to be stored in a format that best represents its data without requiring uniform schemas across all records. As a result, EcoMart can efficiently manage diverse data types while maintaining scalability, flexibility, and ease of data retrieval.
4. Within the proposed MongoDB database solution, EcoMart's business data will be organized into multiple collections based on product category to support efficient storage and retrieval. One collection will store cosmetic product documents, each containing detailed attributes such as brand, product name, price, product ranking,

ingredient lists, and skin-type compatibility indicators. A separate collection will store grocery transaction documents that include member identifiers, purchase dates, and item descriptions. Organizing the data in this way allows EcoMart to accommodate the differing structures of cosmetic and grocery data while maintaining a flexible and scalable design. Storing related information within individual documents eliminates the need for complex joins, enabling faster query performance. EcoMart can use this database solution to retrieve products based on attributes such as brand, price, rankings, or ingredients, as well as analyze grocery purchasing patterns over time to identify trends and optimize inventory decisions. This approach supports efficient data management, enhances analytical capabilities, and improves overall platform performance as the marketplace continues to grow.

- B. The proposed MongoDB database design addresses EcoMart’s scalability concerns by supporting horizontal scaling and efficient data distribution as the platform grows.

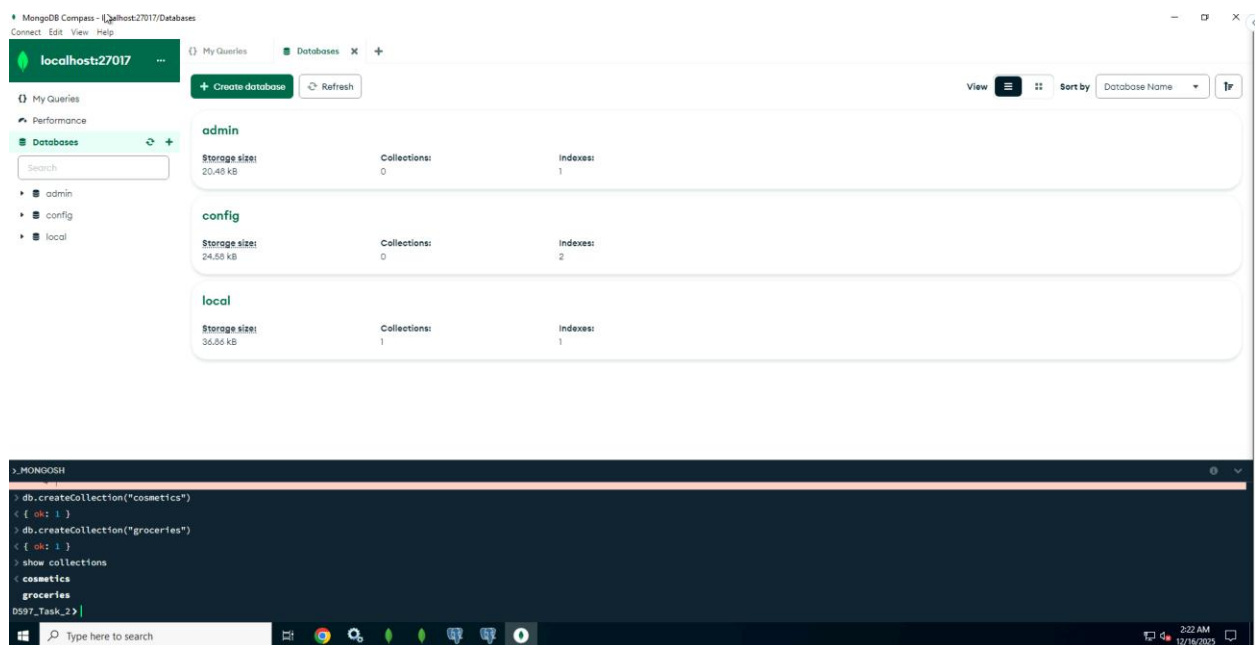
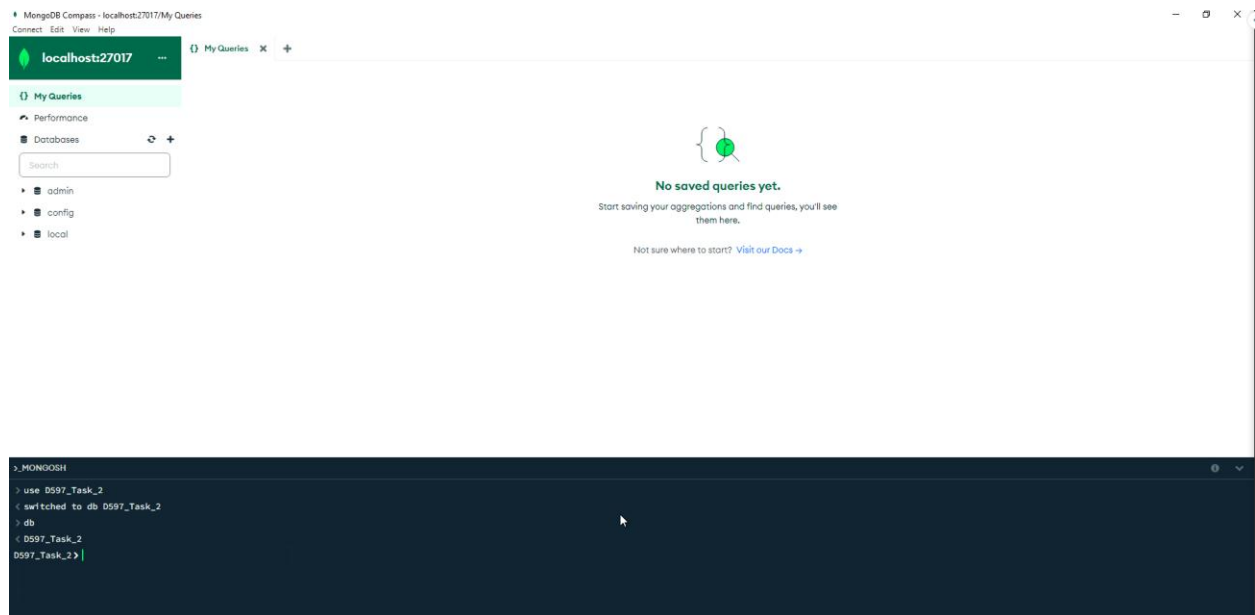
MongoDB is designed to scale horizontally through sharding, which allows data to be distributed across multiple servers rather than relying on a single machine. According to MongoDB, sharding allows databases to “handle large data sets and high throughput operations by distributing data across multiple machines” (MongoDB, 2024). This approach enables EcoMart to handle increasing volumes of product and transaction data without degrading system performance. As user traffic and data storage requirements increase, additional servers can be added to the database cluster to maintain responsiveness and reliability. By leveraging MongoDB’s document-based architecture and built-in scalability features, EcoMart can ensure consistent performance while supporting business expansion and increased user demand.

C. To protect sensitive business and customer data, EcoMart’s proposed MongoDB database design incorporates multiple privacy and security measures. Authentication mechanisms will be implemented to verify user identities and restrict access to authorized personnel only. Role-based access control will be used to ensure users have the minimum level of access required to perform their job functions, reducing the risk of unauthorized data exposure. According to MongoDB, effective database security requires “authentication, authorization, encryption, and auditing controls to protect sensitive data” (MongoDB, 2024). Additionally, encryption will be applied to data both at rest and in transit to protect information from interception or unauthorized access. Regular data backups and monitoring processes will further enhance security by ensuring data integrity and enabling recovery in the event of data loss or system failure. Together, these measures help safeguard EcoMart’s data, support compliance with data privacy standards, and maintain trust with users as the platform continues to grow.

Part 2: Implementation

D.

1.



2. Shows the successful insertion of the groceries dataset into the MongoDB database using the insertMany () method. The script reads the groceries JSON file from the local file system and maps the records into the groceries collection. The acknowledgement message and generated ObjectIds confirm that the data was successfully inserted into the

database.

MongoDB Compass - localhost:27017/D697_Task_2

localhost:27017

My Queries | Databases | D697_Task_2

+ Create collection | Refresh

View | Sort by: Collection Name

cosmetics

Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
4.10 KB	0	0 B	1	4.10 KB

groceries

Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
4.10 KB	0	0 B	1	4.10 KB

```
> use D697_Task_2
> db.cosmetics.insertMany(
  JSON.parse(
    require("fs").readFileSync(
      "C:/Users/Corrine/Documents/Task2Scenario2 Dataset 1_cosmetics.json",
      "utf8"
    )
  )
)
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("6940c464ad13911d814259ff"),
    '1': ObjectId("6940c464ad13911d81425a00"),
    '2': ObjectId("6940c464ad13911d81425a01"),
    '3': ObjectId("6940c464ad13911d81425a02"),
    '4': ObjectId("6940c464ad13911d81425a03"),
    '5': ObjectId("6940c464ad13911d81425a04"),
    '6': ObjectId("6940c464ad13911d81425a05")
  }
}
```

MongoDB Compass - localhost:27017/D697_Task_2

localhost:27017

My Queries | Databases | D697_Task_2

+ Create collection | Refresh

View | Sort by: Collection Name

cosmetics

Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
4.10 KB	0	0 B	1	4.10 KB

groceries

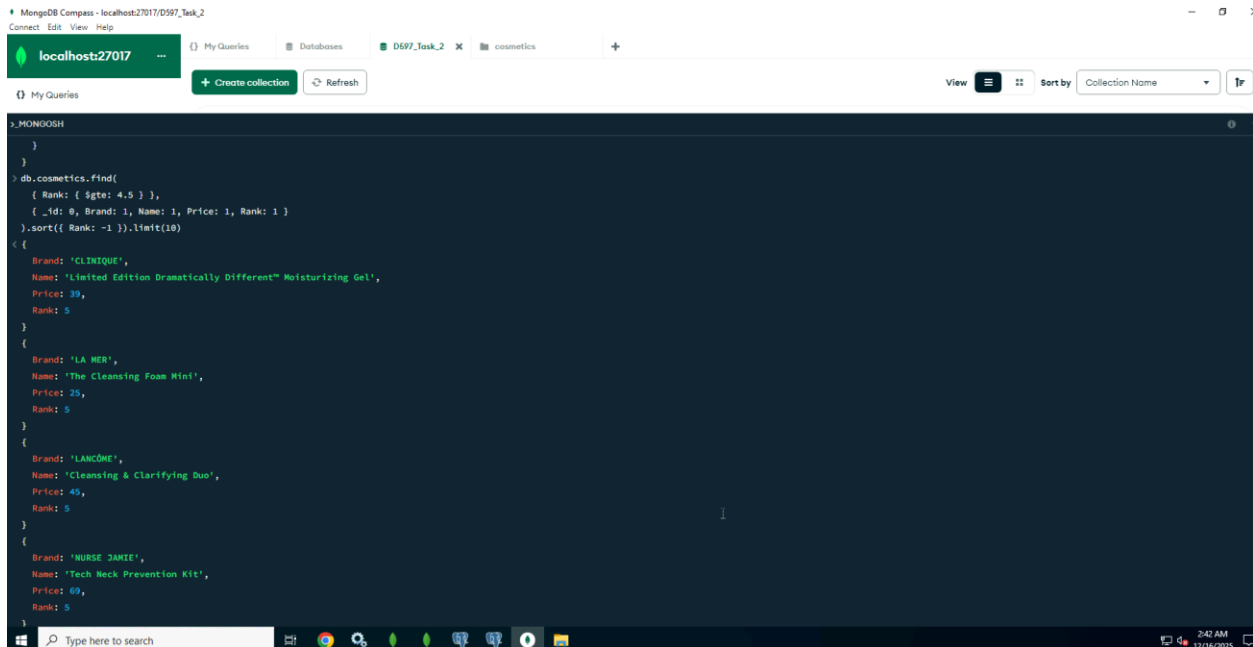
Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
4.10 KB	0	0 B	1	4.10 KB

```
> use D697_Task_2
> db.groceries.insertMany(
  JSON.parse(
    require("fs").readFileSync(
      "C:/Users/Corrine/Documents/Task2Scenario2 Dataset 2_Groceries_dataset.json",
      "utf8"
    )
  )
)
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("6940c464ad13911d81425f00"),
    '1': ObjectId("6940c464ad13911d81425f01"),
    '2': ObjectId("6940c464ad13911d81425f02"),
    '3': ObjectId("6940c464ad13911d81425f03"),
    '4': ObjectId("6940c464ad13911d81425f04")
  }
}
```

3. Query creation

Query 1: High-rated cosmetics products

Query 1 retrieves cosmetic products with a customer rating of 4.5 or higher, sorted by rank. This query supports EcoMart's business objective of identifying top-performing products that can be promoted to customers to improve satisfaction and sales performance.



The screenshot shows the MongoDB Compass interface. The top bar indicates the connection to 'localhost:27017'. The 'Databases' tab is selected, showing a database named 'D597_Task_2' with a collection named 'cosmetics'. The 'My Queries' tab is active, displaying a query that filters for products with a rank of 4.5 or higher and sorts them by rank in descending order. The query is as follows:

```
> MONOOSH
{
  }
}
> db.cosmetics.find(
  { Rank: { $gte: 4.5 } },
  { _id: 0, Brand: 1, Name: 1, Price: 1, Rank: 1 }
).sort({ Rank: -1 }).limit(10)
< {
  Brand: 'CLINIQUE',
  Name: 'Limited Edition Dramatically Different™ Moisturizing Gel',
  Price: 39,
  Rank: 5
}
{
  Brand: 'LA MER',
  Name: 'The Cleansing Foam Mini',
  Price: 35,
  Rank: 5
}
{
  Brand: 'LANCÔME',
  Name: 'Cleansing & Clarifying Duo',
  Price: 45,
  Rank: 5
}
{
  Brand: 'NURSE JAMIE',
  Name: 'Tech Neck Prevention Kit',
  Price: 69,
  Rank: 5
}
```

The bottom of the screen shows the Windows taskbar with the time 2:42 AM on 12/16/2025.

Query 2: Sensitive-skin friendly items (customer personalization)

Query 2 identifies affordable cosmetics products price under \$30 that also maintains high customer ratings. This supports EcoMart's goal of offering quality products to price-sensitive customers.

The screenshot shows the MongoDB Compass interface. The top bar indicates the connection is to 'localhost:27017'. The left sidebar shows the 'D597Task2' database and the 'cosmetics' collection. The main panel displays a query in the 'My Queries' tab:

```
> MONDOSH
{ Price: 4.6,
  Rank: 5
}
> db.cosmetics.find(
  { Price: { $lte: 30 }, Rank: { $gte: 4.5 } },
  { _id: 0, Brand: 1, Name: 1, Price: 1, Rank: 1 }
).sort({ Price: 1 })
<
{
  Brand: 'SEPHORA COLLECTION',
  Name: 'Soft Touch Cotton Pads',
  Price: 4,
  Rank: 4.6
}
{
  Brand: 'SEPHORA COLLECTION',
  Name: 'Sleeping Mask - Coconut Water',
  Price: 4,
  Rank: 5
}
{
  Brand: 'SEPHORA COLLECTION',
  Name: 'Sleeping Mask - Aloe Vera',
  Price: 4,
  Rank: 5
}
{
  Brand: 'DR. JART+',
  Name: 'DermaMask Water Jet Vital Hydra Solution™',
  Price: 6,
  Rank: 4.6
}
```

Query 3: Most frequently purchased grocery items (demand/inventory)

Query 3 aggregates grocery purchase data to identify the most frequently purchased items. This information supports inventory planning and demand forecasting for EcoMart's grocery operations.

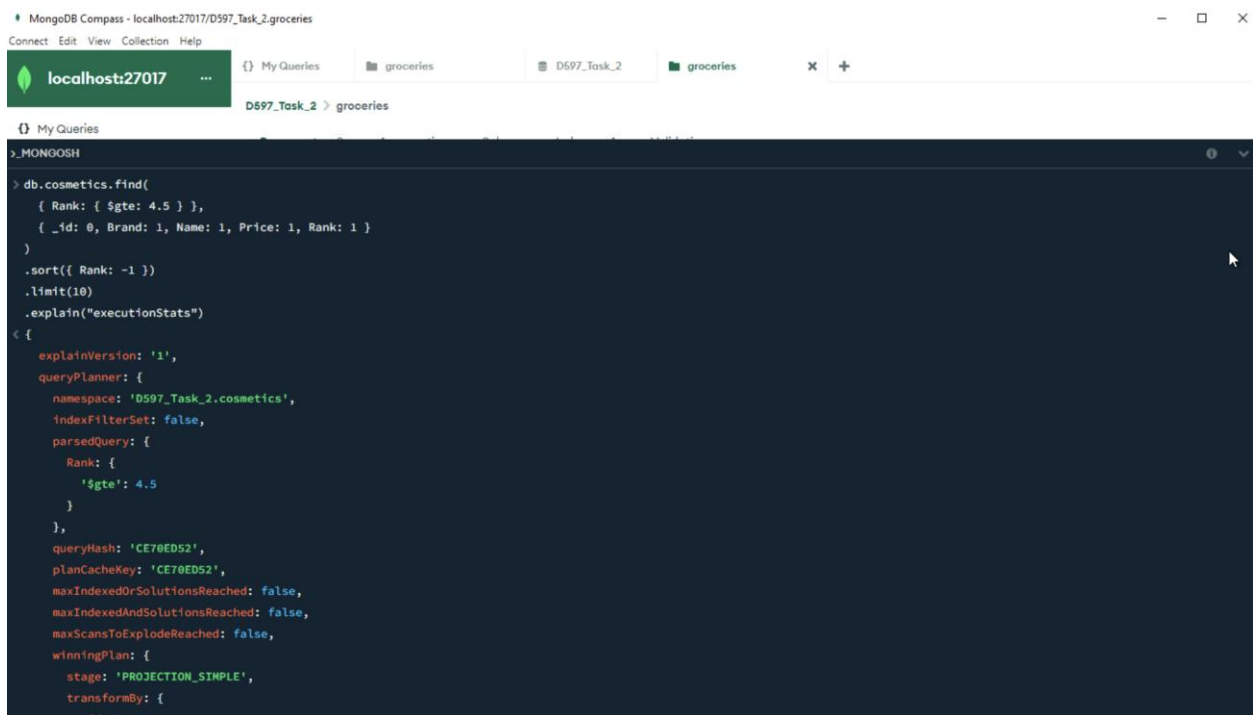
The screenshot shows the MongoDB Compass interface. The top bar indicates the connection is to 'localhost:27017'. The left sidebar shows the 'D597Task2' database and the 'groceries' collection. The main panel displays an aggregation query in the 'My Queries' tab:

```
> MONDOSH
> use D597Task2
db.groceries.aggregate([
  { $match: { itemDescription: /milk/i } },
  { $group: { _id: "$itemDescription", purchaseCount: { $sum: 1 } } },
  { $sort: { purchaseCount: -1 } },
  { $limit: 10 }
])
< already on db D597Task2
> db.groceries.aggregate([
  { $match: { itemDescription: /milk/i } },
  { $group: { _id: "$itemDescription", purchaseCount: { $sum: 1 } } },
  { $sort: { purchaseCount: -1 } },
  { $limit: 10 }
])
<
{
  _id: 'whole milk',
  purchaseCount: 2592
}
{
  _id: '1%LT-milk',
  purchaseCount: 323
}
{
  _id: 'butter milk',
  purchaseCount: 263
}
{
  _id: 'condensed milk',
  purchaseCount: 98
}
}
```

4. Optimization

Query 1: Pre-Optimization

Prior to optimization, Query 1 executed in 30 milliseconds. The query plan used a (COLLSCAN), requiring MongoDB to examine a larger number of documents to return the results.



The screenshot shows the MongoDB Compass interface. The top bar indicates the connection to 'localhost:27017' and the database 'D597_Task_2'. The 'My Queries' tab is active, showing a query for the 'cosmetics' collection. The query is as follows:

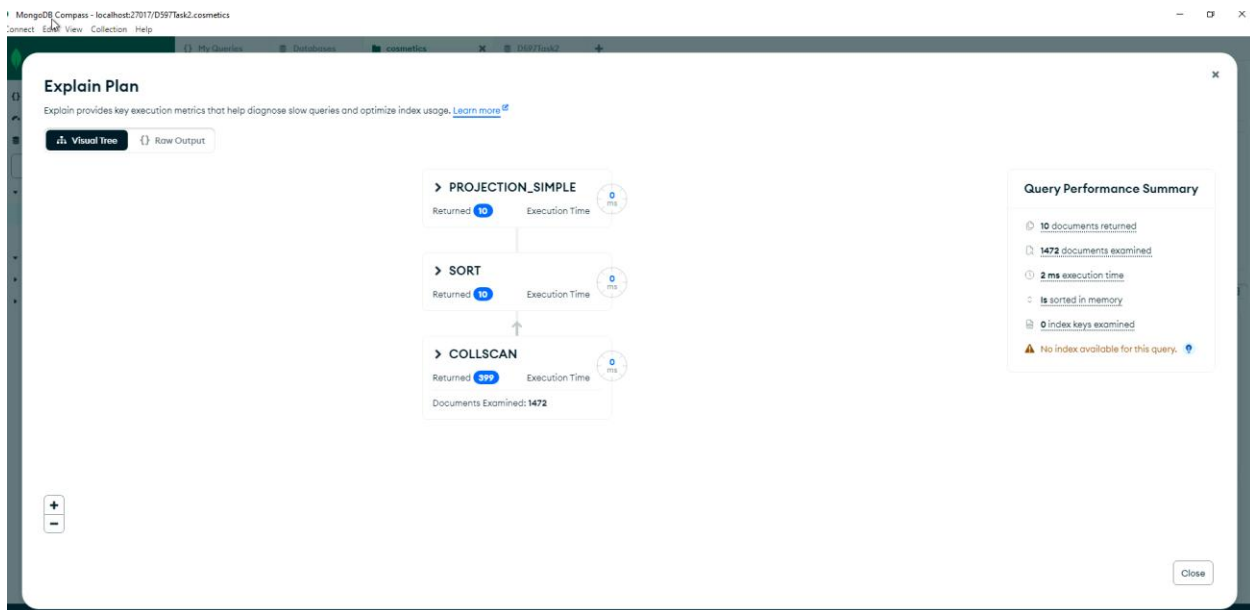
```
> db.cosmetics.find(
  { Rank: { $gte: 4.5 } },
  { _id: 0, Brand: 1, Name: 1, Price: 1, Rank: 1 }
)
.sort({ Rank: -1 })
.limit(10)
.explain("executionStats")
```

The execution plan is displayed below the query, showing the following details:

```
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'D597_Task_2.cosmetics',
    indexFilterSet: false,
    parsedQuery: {
      Rank: {
        '$gte': 4.5
      }
    },
  },
  queryHash: 'CE70ED52',
  planCacheKey: 'CE70ED52',
  maxIndexedOrSolutionsReached: false,
  maxIndexedAndSolutionsReached: false,
  maxScansToExplodeReached: false,
  winningPlan: {
    stage: 'PROJECTION_SIMPLE',
    transformBy: {
```

```
>_MONGOSH
{
  limitAmount: 10,
  type: 'simple',
  inputStage: {
    stage: 'COLLSCAN',
    filter: {
      Rank: {
        '$gte': 4.5
      }
    },
    direction: 'forward'
  }
},
rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 10,
  executionTimeMillis: 30,
  totalKeysExamined: 0,
  totalDocsExamined: 1472,
  executionStages: {
    stage: 'PROJECTION_SIMPLE',
    nReturned: 10,
    executionTimeMillisEstimate: 24,
    works: 1484,

```



Query 2: Pre-Optimization

Prior to optimization, Query 2 executed in 4 milliseconds. At this stage, the query plan relied on a COLLSCAN, as no supporting index existed for the filtering and sorting conditions.

Compass

My Queries

Data Modeling

CONNECTIONS (2)

Search connections

localhost:27017

D597_Task_2

cosmetics

groceries

admin

config

local

localhost:27017

mongosh: localhost:27017

mongosh: localhost:27017

mongosh: localhost:27017

>_MONGOSH

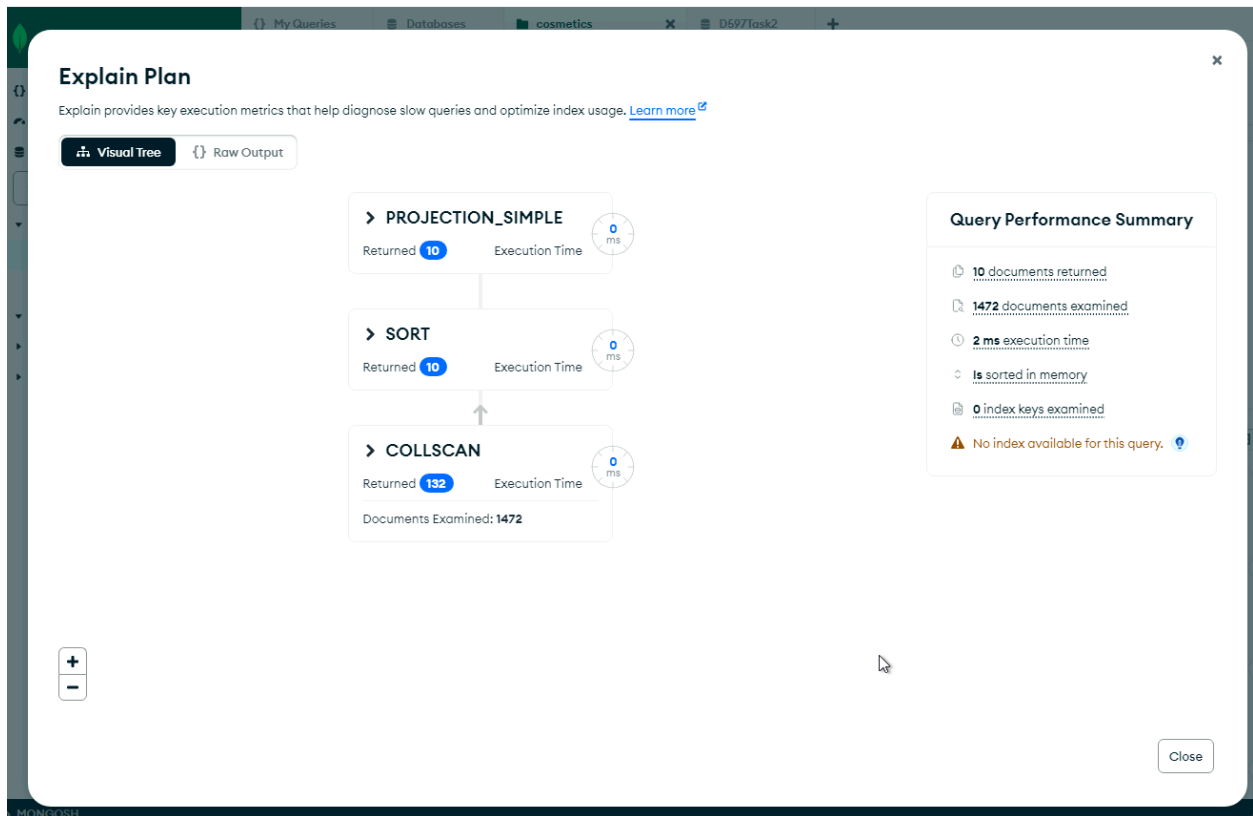
mongosh: localhost:27017

```
> use D597_Task_2
< switched to db D597_Task_2
> db.cosmetics.find(
  { Price: { $lte: 30 }, Rank: { $gte: 4.5 } },
  { _id: 0, Brand: 1, Name: 1, Price: 1, Rank: 1 }
)
.sort({ Price: 1 })
.explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'D597_Task_2.cosmetics',
    indexFilterSet: false,
    parsedQuery: {
      '$and': [
        {
          Price: {
            '$lte': 30
          }
        },
        {
          Rank: {
            '$gte': 4.5
          }
        }
      ]
    },
    queryHash: 'D1A06AE0',
    planCacheKey: 'D1A06AE0',
    planId: 1,
    stage: 'COLLSCAN',
    filter: {
      '$and': [
        {
          Price: {
            '$lte': 30
          }
        },
        {
          Rank: {
            '$gte': 4.5
          }
        }
      ]
    },
    direction: 'forward'
  },
  rejectedPlans: []
},
  executionStats: {
    executionSuccess: true,
    nReturned: 132,
    executionTimeMillis: 4,
    totalKeysExamined: 0
  }
}
```

>_MONGOSH

mongosh: localhost:27017

```
Rank: 1
},
inputStage: {
  stage: 'COLLSCAN',
  filter: {
    '$and': [
      {
        Price: {
          '$lte': 30
        }
      },
      {
        Rank: {
          '$gte': 4.5
        }
      }
    ]
  },
  direction: 'forward'
}
},
rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 132,
  executionTimeMillis: 4,
  totalKeysExamined: 0
}
```



Query 3: Pre-Optimization

Prior to optimization, Query 3 took 135 milliseconds to run (using `explain("executionStats")`). The query plan relied on a COLLSCAN, requiring MongoDB to examine the entire groceries collection to compute grouped purchase totals. Since no supporting index was available on the `itemDescription` field, the aggregation incurred higher execution time due to full document scanning.

```
>_MONGOOSH
```

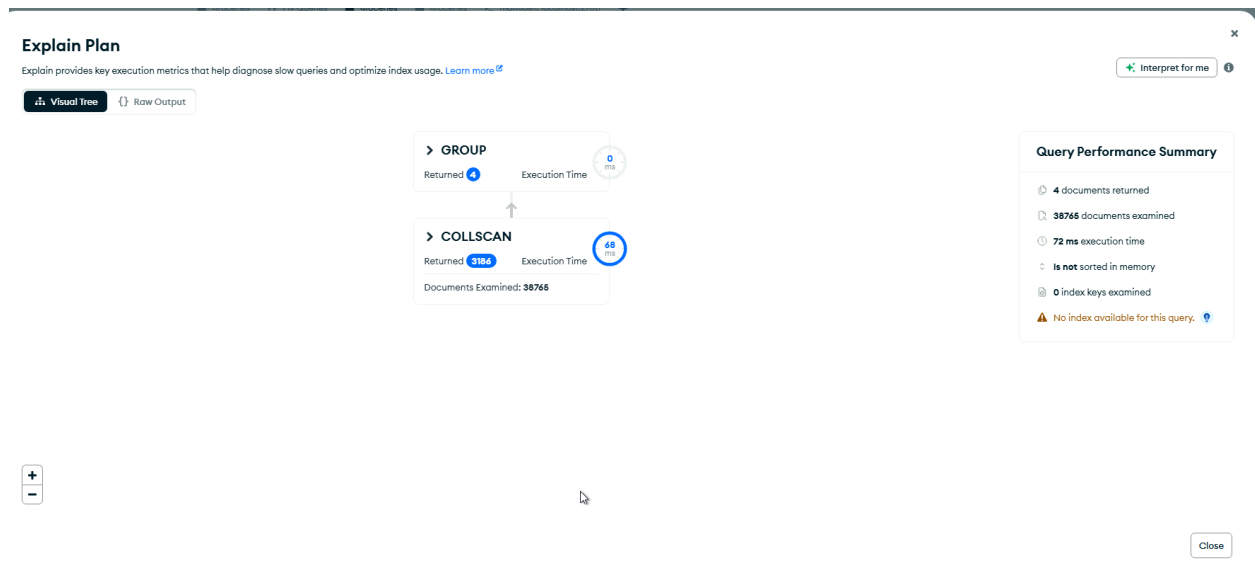
```
1
```

```
> db.groceries.aggregate([
  { $match: { itemDescription: /milk/i } },
  { $group: { _id: "$itemDescription", purchaseCount: { $sum: 1 } } },
  { $sort: { purchaseCount: -1 } },
  { $limit: 10 }
]).explain("executionStats")
< {
  explainVersion: '2',
  stages: [
    {
      '$cursor': {
        queryPlanner: {
          namespace: 'D597Task2.groceries',
          indexFilterSet: false,
          parsedQuery: {
            itemDescription: BSONRegExp('milk', 'i')
          },
          queryHash: 'EEA1D686',
          planCacheKey: '19696514',
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExploreReached: false,
          winningPlan: {
            queryPlan: {
              stage: 'GROUP',
              planNodeId: 3,
              inputStage: {
                stage: 'COLLSCAN',
                planNodeId: 1,
                filter: {
                  itemDescription: BSONRegExp('milk', 'i')
                },
              },
            },
            direction: 'forward'
          }
        }
      }
    }
  ]
}
```

```
>_MONGOOSH
```

```
stages: [
  {
    '$cursor': {
      queryPlanner: {
        namespace: 'D597Task2.groceries',
        indexFilterSet: false,
        parsedQuery: {
          itemDescription: BSONRegExp('milk', 'i')
        },
        queryHash: 'EEA1D686',
        planCacheKey: '19696514',
        maxIndexedOrSolutionsReached: false,
        maxIndexedAndSolutionsReached: false,
        maxScansToExploreReached: false,
        winningPlan: {
          queryPlan: {
            stage: 'GROUP',
            planNodeId: 3,
            inputStage: {
              stage: 'COLLSCAN',
              planNodeId: 1,
              filter: {
                itemDescription: BSONRegExp('milk', 'i')
              },
            },
            direction: 'forward'
          }
        }
      }
    }
  ],
  rejectedPlans: []
],
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 135,
  totalKeysExamined: 0,
  totalDocsExamined: 38765,
  executionStages: {
    stage: 'mkobj',
    planNodeId: 3,
    nReturned: 4,
    executionTimeMillisEstimate: 130,
    opens: 1,
    closes: 1,
    saveState: 39,
    restoreState: 39,
    isEOF: 1,
    objSlot: 12,
    fields: [],
    projectFields: [
      '_id',
      'purchaseCount'
    ],
    projectSlots: [
      9,

```



Query 1: Indexing & Post-Optimization

For Query 1, an index was created on the Rank field to optimize ranking-based filtering and sorting. After the index was created, Query 1 executed in 6 milliseconds, utilizing an index scan (IXSCAN), which reduced the number of documents and improved overall query performance.

```
ok: 1
}
> db.cosmetics.createIndex({ Rank: -1 })
< Rank_-1
D597_Task_2>
```

localhost:27017

My QueriesPerformanceDatabases

D597_Task_2cosmeticsgroceries

D597_Task_2 > cosmetics

Documents 0AggregationsSchemaIndexes 1Validation

Create IndexRefresh

VIEWINGINDEXESSearch Indexes

Name and Definition	Type	Size	Usage	Properties
> _id_	REGULAR ⓘ	28.7 KB	6 (since Tue Dec 30 2025)	UNIQUE ⓘ
> Rank_-1	REGULAR ⓘ	28.7 KB	0 (since Tue Dec 30 2025)	

adminconfiglocal

```
Documents 0 Aggregations Schema Indexes 1 Validation
> MONGODB
OK: 1
}
> db.cosmetics.createIndex({ Rank: -1 })
< Rank_-1
> db.cosmetics.find(
  { Rank: { $gte: 4.5 } },
  { _id: 0, Brand: 1, Name: 1, Price: 1, Rank: 1 }
)
.sort({ Rank: -1 })
.limit(10)
.explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'D597_Task_2.cosmetics',
    indexFilterSet: false,
    parsedQuery: {
      Rank: {
        '$gte': 4.5
      }
    },
    queryHash: 'CE70ED52',
    planCacheKey: 'B4E2C9DA',
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    maxScannedKeysReached: false
  },
  executionStats: {
    executionTimeMs: 0.0001,
    indexesUsed: [
      Rank_-1
    ],
    keysExamined: 1,
    documentsExamined: 1,
    documentsReturned: 1
  },
  profile: 0
}
```

```
Documents 0 Aggregations Schema Indexes 1 Validation
My Queries
> MONGODB
{
  inputStage: {
    stage: 'FETCH',
    inputStage: {
      stage: 'IXSCAN',
      keyPattern: {
        Rank: -1
      },
      indexName: 'Rank_-1',
      isMultiKey: false,
      multiKeyPaths: {
        Rank: []
      },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        Rank: [
          '[inf.0, 4.5]'
        ]
      }
    }
  },
  stage: 'FETCH',
  keysExamined: 1,
  documentsExamined: 1,
  documentsReturned: 1
}
```

```
>_MONGOSH
}
}
}
}
},
rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 10,
  executionTimeMillis: 0,
  totalKeysExamined: 10,
  totalDocsExamined: 10,
  executionStages: {
    stage: 'LIMIT',
    nReturned: 10,
    executionTimeMillisEstimate: 0,
    works: 11,
    advanced: 10,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    limitAmount: 10,
    inputStage: {
```

Query 2: Indexing & Post-Optimization

A compound index on Price and Rank was created to support filtering and sorting requirements

Query 2 executed in 7 milliseconds. Although execution time varied slightly between runs, the query plan changed from COLLSCAN to IXSCAN after creating the index, confirming the query is using the index and demonstrating optimization.

```
}
> db.cosmetics.createIndex({ Price: 1, Rank: 1 })
< Price_1_Rank_1
D597_Task_2>
```

Compass

My Queries

Data Modeling

CONNECTIONS (2)

Search connections

localhost:27017

D597_Task_2

cosmetics

groceries

admin

config

local

localhost:27017

localhost:27017

localhost:27017

localhost:27017

cosmetics

localhost:27017 > D597_Task_2 > cosmetics

Open MongoDB shell

Documents 1.5K Aggregations Schema **Indexes 2** Validation

Create Index Refresh

VIEWING

INDEXES

SEARCH INDEXES

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR	28.7 kB	1 (since Tue Dec 30 2025)	UNIQUE	READY
> Price_1_Rank_1	REGULAR	41.0 kB	0 (since Tue Dec 30 2025)	COMPOUND	READY



localhost:27017 localhost:27017 localhost:27017 cosmetics localhost:27017

```
> MONGOSH
ok: 1
}
> db.cosmetics.createIndex({ Price: 1, Rank: 1 })
< Price_1_Rank_1
> db.cosmetics.find(
  { Price: { $lte: 30 }, Rank: { $gte: 4.5 } },
  { _id: 0, Brand: 1, Name: 1, Price: 1, Rank: 1 }
)
.sort({ Price: 1 })
.explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'D597_Task_2.cosmetics',
    indexFilterSet: false,
    parsedQuery: {
      '$and': [
        {
          Price: {
            '$lte': 30
          }
        },
        {
          Rank: {
            '$gte': 4.5
          }
        }
      ]
    }
  },
}
```

>_ mongosh: localhost:27017

>_ mongosh: localhost:27017

>_ mongosh: localhost:27017

cosmetics

>_ mongosh: localhost:27017

+

>_MONGOSH

1

```
    },
    inputStage: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: {
          Price: 1,
          Rank: 1
        },
      },
      indexName: 'Price_1_Rank_1',
      isMultikey: false,
      multiKeyPaths: {
        Price: [],
        Rank: []
      },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        Price: [
          '[-inf.0, 30]'
        ],
        Rank: [
          '[4.5, inf.0]'
        ]
      }
    }
  }
}
```

>_MONGOSH

1

```
    '[4.5, inf.0]'
  ]
}
}
},
rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 132,
  executionTimeMillis: 7,
  totalKeysExamined: 160,
  totalDocsExamined: 132,
  executionStages: {
    stage: 'PROJECTION_SIMPLE',
    nReturned: 132,
    executionTimeMillisEstimate: 0,
    works: 160,
    advanced: 132,
    needTime: 27,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    transformBy: {
      _id: 0,
      Brand: 1,
      Name: 1,
    }
  }
}
```

Query 3: Indexing & Post-Optimization

An index on the `itemDescription` field was created to support keyword-based filtering within the aggregation pipeline. Query 3 executed in 134ms. Although execution time remained relatively consistent due to the required aggregation and grouping operations, the query plan changed from `COLLSCAN` to `IXSCAN` after creating the index. This confirms that MongoDB utilized the index to efficiently filter matching documents prior to aggregation, demonstrating query execution and effective indexing.

```
> db.groceries.createIndex({ itemDescription: 1 })
< itemDescription_1
```

localhost:27017 > D597Task2 > groceries Open MongoDB shell

Documents 39K Aggregations Schema **Indexes 1** Validation

Create Index Refresh

VIEWING INDEXES SEARCH INDEXES

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR	397.3 kB	2 (since Sun Jan 04 2026)	UNIQUE	READY
> itemDescription_1	REGULAR	249.9 kB	1 (since Sun Jan 04 2026)		READY

```
> MONGOSH
< itemDescription_1
> db.groceries.aggregate([
  { $match: { itemDescription: /milk/i } },
  { $group: { _id: "$itemDescription", purchaseCount: { $sum: 1 } } },
  { $sort: { purchaseCount: -1 } },
  { $limit: 10 }
]).explain("executionStats")
< {
  explainVersion: '2',
  stages: [
    {
      '$cursor': {
        queryPlanner: {
          namespace: 'D597Task2.groceries',
          indexFilterSet: false,
          parsedQuery: {
            itemDescription: BSONRegExp('milk', 'i')
          },
          queryHash: 'EEA1D606',
          planCacheKey: '9260E28E',
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            queryPlan: {
              stage: 'GROUP',
              planNodeId: 3,
              inputStage: {
                stage: 'PROJECTION_COVERED',
                planNodeId: 2,
                transformBy: {
                  itemDescription: true,
                  _id: false
                }
              }
            }
          }
        }
      }
    }
  ]
}
```


Panopto Presentation Link

<https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=410e8f93-e642-46a5-b786-b3c8017c11b5>

References

MongoDB, Inc. (2024). *Database scaling*. <https://www.mongodb.com/basics/scaling>

MongoDB, Inc. (2024). *NoSQL databases explained*.

<https://www.mongodb.com/resources/basics/databases/nosql-explained>

MongoDB, Inc. (2024). *Security checklist for self-managed deployments*.

<https://www.mongodb.com/docs/manual/administration/security-checklist/>